

Universidade do Minho
Escola de Engenharia

Apresentação do Projeto de Engenharia Web

Plataforma de Gestão e Disponibilização de Recursos Educativos

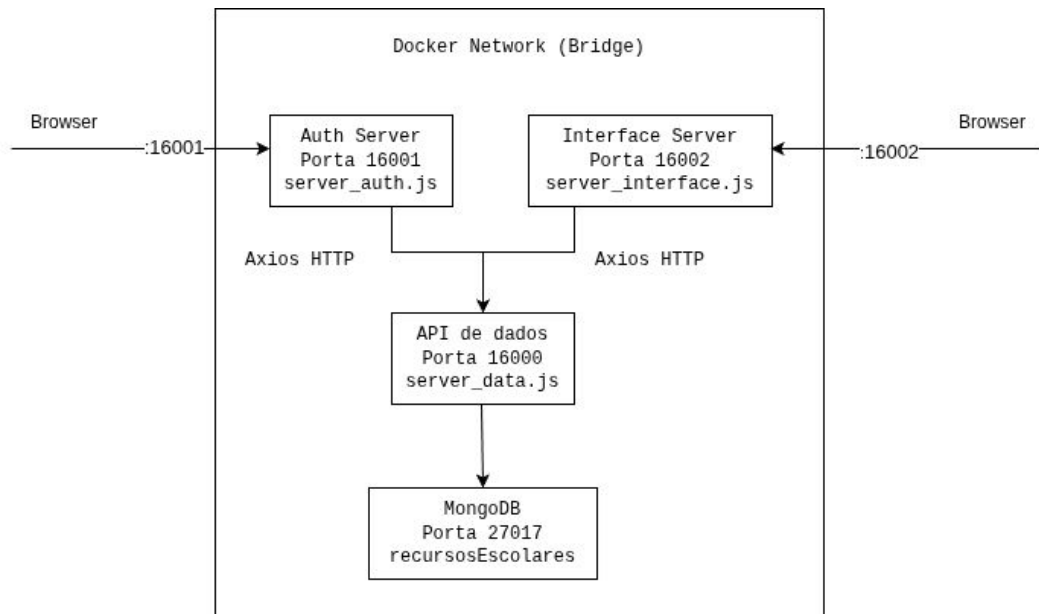
11 de Maio 2026

Estrutura da apresentação

- Arquitetura do projeto
- Modelo da base de dados
- API de dados
- Serviço de autenticação
- Serviço de interface
- Demonstração

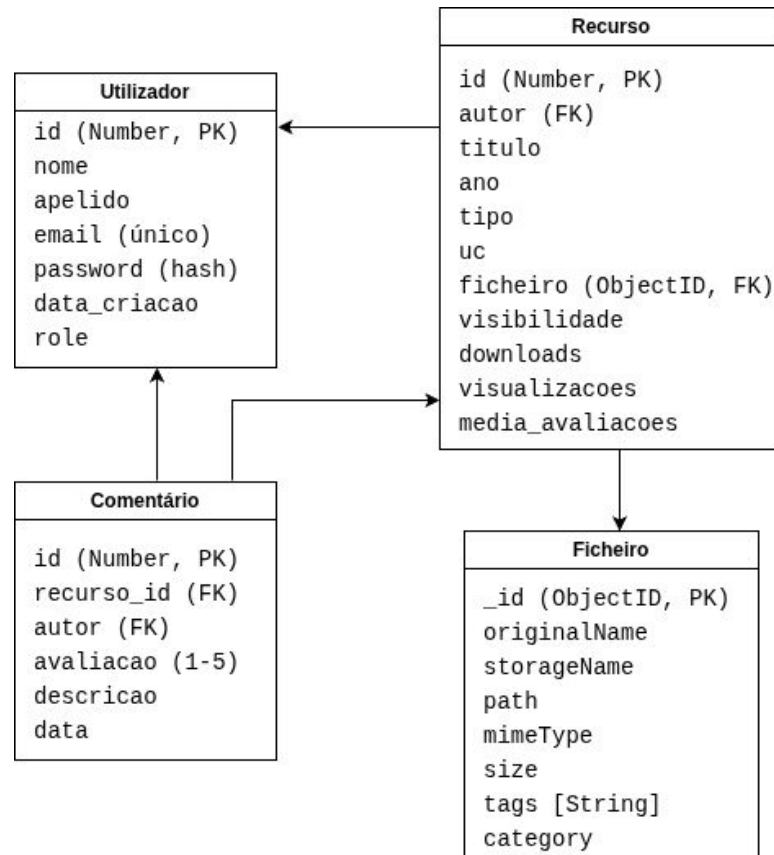
Arquitetura do projeto

- Arquitetura de microserviços com 3 servidores independentes (**API de dados, autenticação e interface**).
- Cada serviço tem a sua responsabilidade, e respectivos **Dockerfile** e **package.json**.
- A comunicação entre serviços é feita através de pedidos **HTTP**.
- A aplicação é montada com recurso a um **Docker Compose**.



Modelo da base de dados

- A aplicação dispõe de quatro entidades, **Recurso**, **Comentário**, **Utilizador** e **Ficheiro**.
- Em algumas das entidades (Recurso, Utilizador e Ficheiro) foram definidos **índices** para facilitar a pesquisa.



API de dados

- Desenvolvemos uma **REST API** com os métodos **GET**, **POST**, **PUT** e **DELETE** para as várias entidades.
- A API comunica com o MongoDB através do pacote **mongoose**.
- O upload de ficheiros é feito através do **multer**.
- O armazenamento dos ficheiros é organizado pela UC associada ao recurso, ficando com uma estrutura de pastas deste género:

`/uploads/{uc}/{timestamp}-{hash}.ext`

- Documentação da API com Swagger UI disponível em:

`http://localhost:16000/api-docs`

API de dados - exemplos de endpoints

Método	Rota	Descrição
GET	/api/recursos	Listar
GET	/api/recursos/:id	Obter por id
POST	/api/recursos	Criar
PUT	/api/recursos/:id	Atualizar
DELETE	/api/recursos/:id	Apagar

Exemplos de parâmetros para a **Query String** disponíveis:

- q=texto
- tipo=Slides
- ano=2023-2024
- uc=DSS
- visibilidade=publico
- _sort=data_resgisto&_order=asc
- _select=ano,tipo,uc

Serviço de autenticação

- Servidor express independente à escuta na porta 16001.
- **Responsabilidade única:** gerir a identidade do utilizador e as suas credenciais.
- A **lógica de segurança** está centralizada neste módulo.
- Renderiza as páginas de login registo definidas com **templates Pug**

Fluxo de autenticação

1. Utilizador acede → Auth Server (16001)
|
▼
2. POST /login
 - └ Consulta API de Dados → GET /api/users?email=X
 - └ Verifica password com `bcrypt.compare()`
 - └ Gera JWT assinado (payload: id, email, nome, apelido, role)
|
▼
3. Cookie HttpOnly com JWT é definido

|
▼
4. Redireciona para Interface (16002)

Serviço de autenticação - Tokens JWT e passwords

Os tokens JWT têm um payload com o formato apresentado a seguir. Desta forma é possível termos sempre presente o **id** e **role** do utilizador que são usados para dar **permissões de acesso** a diversas rotas e funcionalidades.

```
// Payload do token
{
    id:          Number,
    email:       String,
    nome:        String,
    apelido:     String,
    role:        String
}
```

Para sermos mais realistas e termos alguma **segurança** implementada, o registo de passwords implica que uma password tenha pelo menos 8 caracteres, dos quais 1 letra maiúscula, 1 letra minúscula e 1 dígito.

As passwords não são diretamente guardadas na base de dados, mas sim um **hash** da password.

Serviço de interface

- Servidor express à escuta na porta 16002.
- Renderiza **HTML** com recurso a **templates Pug**.
- Consome dados através da API.
- **Verifica a autenticação** com o JWT do cookie (partilha o segredo com o serviço de autenticação).
- Implementa um **controlo de acesso** baseado em roles (RBAC).

Permissões (RBAC):

consumidor (nível 1)

- ├ Ver recursos públicos
- └ Comentar recursos

produtor (nível 2)

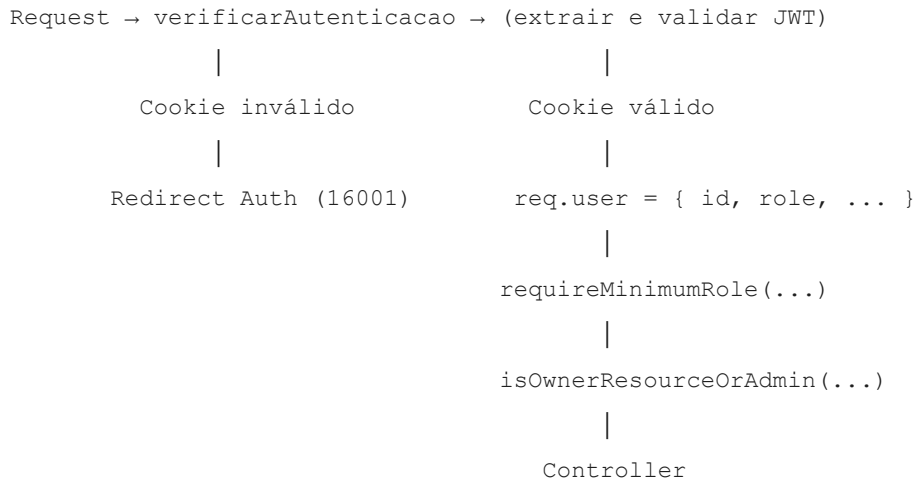
- ├ Tudo do consumidor
- ├ Criar recursos
- └ Editar / eliminar os seus próprios recursos

admin (nível 3)

- ├ Tudo do produtor
- ├ Editar / eliminar qualquer recurso
- └ Gerir utilizadores (roles, eliminação)

Serviço de interface - Middlewares

- **verificaAutenticacao:** Valida o JWT do cookie.
- **requireMinimumRole:** Permite o acesso mediante o role.
- **isOwnerOrAdmin:** Permite o acesso ao dono do recurso ou ao administrador.
- **isOwnerProfile:** Permite editar apenas o próprio perfil.
- **canViewResource:** Permite o dono ou o administrador ver um recurso privado.



Demonstração